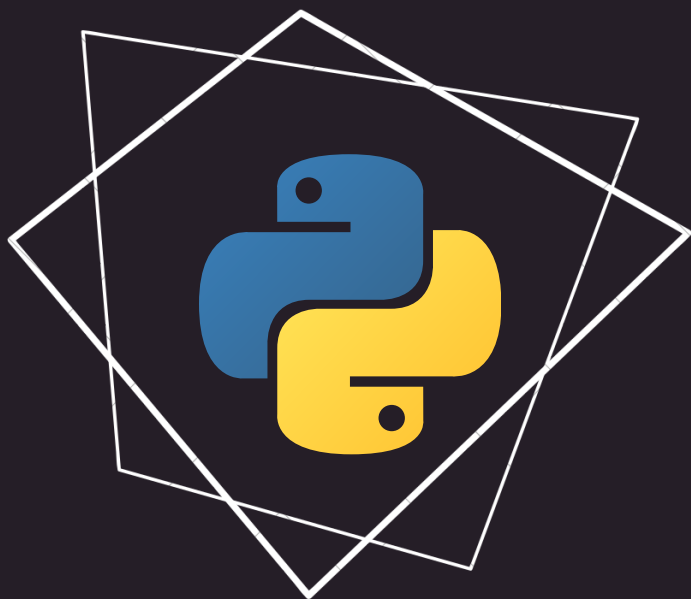


# APRENDENDO PYTHON DO JEITO MAIS FÁCIL



Higor Silva  
Lopes Granero

Primeira  
Edição



---

# CONTEÚDO

## CAPÍTULO 1:

- Sobre o Autor: Quem Sou Eu ?
- Introdução
- O que é Python ?
- Por que Python ?
- O que podemos fazer com Python ?
- Como instalar Python
  - Windows
  - MacOs
  - Linux

## CAPÍTULO 2:

- Sobre o Autor: Quem Sou Eu ?
- Introdução
- O que é Python ?
- Por que Python ?
- O que podemos fazer com Python ?

## CAPÍTULO 3:

- Variáveis e Operadores
  - Nomenclatura de Variáveis
  - Atribuição
  - Operações Básicas
  - Mais Algumas Operações
- 
-

# CONTEÚDO

## CAPÍTULO 4:

- Tipo de Dados em Python
- O que são e o que não são números.
- Manipuladores de Strings
- O tipo booleano
- Exibindo Mensagens Concatenadas em Python
- Conversores

## CAPÍTULO 5:

- Listas em Python
- Tuplas
- Dicionários

## CAPÍTULO 6:

- Programa Interativo com Python
- Aspas Triplas
- Cortar/Pular Linha no Python

## CAPÍTULO 7:

- Condições em Python
- Diferença entre == & IS
- Diferença entre != & IS NOT
- O Comando IF
- Operadores AND & OR

Bônus: Exercício de Algoritmo



---

# CONTEÚDO

## CAPÍTULO 8:

- Laços de Repetição
- Comando FOR
- Laços de Repetição através de uma sequência de números.
- Comando WHILE
- Comando BREAK
- Comando CONTINUE

## CAPÍTULO 9:

- Try & Except

## CAPÍTULO 10:

- O Que São Funções ?
- Definindo Uma Função
- Funções Prontas
- Módulos
- Matemática em Python (Math)
- Datas em Python(Datetime)
- Criando um Módulo

## CAPÍTULO 11:

- Exercícios Resolvidos

## CAPÍTULO 12:

- Desafios
- 

# QUEM SOU EU ?



**Higor Granero é programador sênior e já trabalhou para empresas como:**

- Itaú
- XP Investimentos
- Votorantim
- Raízen
- Dori
- Faber Castell
- Itambé
- Riachuelo
- Veracel
- Brasken
- Cerradinho
- Marilan
- Volvo
- Nordesteão
- Bombril
- Veracel
- Bayer

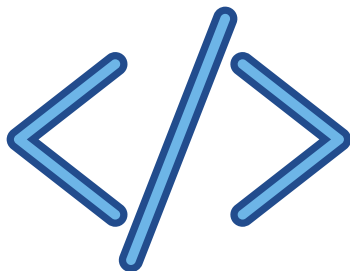
- Bradesco
- Oi
- Petrobrás
- Paranapanema
- Eldorado e etc...

Iniciou sua carreira com tecnologia aos 14 anos de idade, dando sempre ênfase à prática das teorias de todo conteúdo que aprendia, seja redes, hacking(sua maior paixão), ERP(Sistemas para Empresas) e principalmente programação.

Aconselha que todo iniciante saia da camada básica de apenas leitura ou absorção de conhecimento e vá para onde realmente ocorre a diferença, a prática!

Pois acredita que a teoria é apenas a abstração da informação e a prática é a concretização da mesma.

# INTRODUÇÃO



Você está prestes a conhecer uma das linguagens mais poderosas já criadas no **MUNDO**, feita para ser robusta e de fácil aprendizagem, conseguindo realizar as tarefas mais simples até as mais complexas.

Estou muito feliz por você adquirir este livro, aqui está um conhecimento acumulado em mais de 7 anos de experiência e traduzido para aventureiros na área de tecnologia, ou seja, iniciantes e até programadores mais experientes para que consigam entender e **PRATICAR** a magia do **Python**.

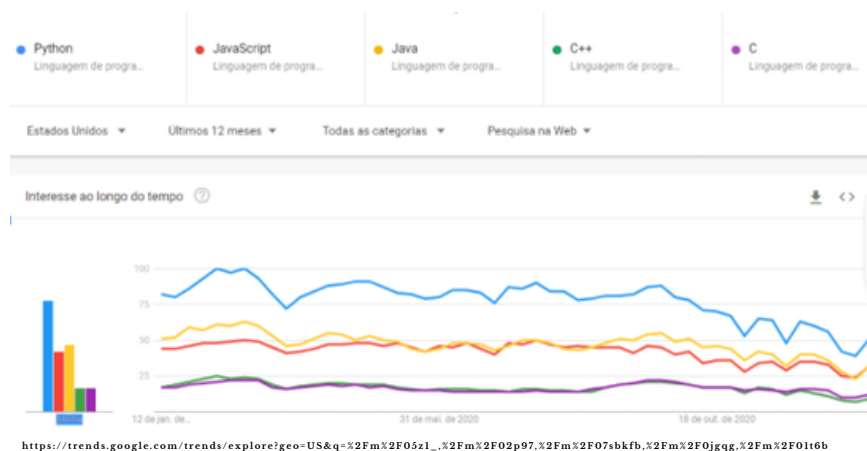
# VAMOS COM CALMA! O QUE É PYTHON ?



Python é uma Linguagem de programação de alto nível, criada no ano de 1989 pelo Holandês Guido Van Rossum, que aproveitando suas férias de trabalho na semana do natal, decidiu colocar em prática o seu “hobbie” de programar, assim criando a ideia que futuramente chamaria-se Python. Como a grande maioria das linguagens de programação de alto nível, ou seja, aquelas linguagens onde estão mais perto do entendimento dos seres humanos, Python é escrita em Inglês e necessita da instalação de um interpretador para o seu funcionamento. Nós veremos a instalação para do Python no Capítulo 2, para os três principais sistemas operacionais: Windows, Linux e MacOS.

# VAMOS COM CALMA! POR QUE PYTHON ?

Python é a linguagem que mais cresce em todo o mundo, segue abaixo um gráfico de quais são as linguagens mais procuradas no google.



Como pode ser visto, a tendência do crescimento do Python é maior do que qualquer outra linguagem, isto porque Python é FÁCIL de APRENDER e ao mesmo tempo MUITO, mas MUITO PODEROSA(Guardem esta palavra!).

# VAMOS COM CALMA! POR QUE PYTHON ?

Tanto para quem está há anos no mercado e já trabalhou com outras linguagens de programação, até para as pessoas que não tiveram nenhuma experiência na área, conseguirão aprender Python sem muitos esforços, além da comunidade, fóruns e discussões sobre Python ser uma das maiores do mundo.



**Welcome to python-forum.io**

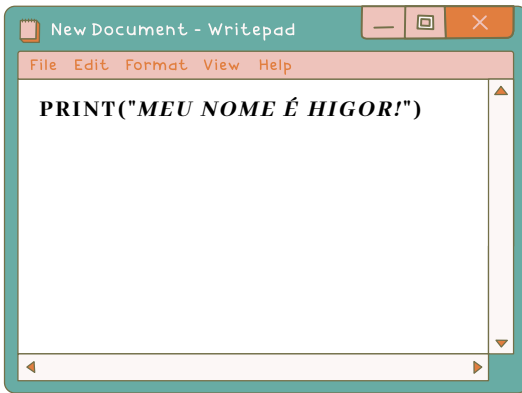
Always look on the forum.io side of  
Python

python-forum.io

## A COMPLEXIDADE DO PYTHON VS OUTRAS LINGUAGENS

Colocarei um exemplo da complexidade de cada linguagem mencionada no gráfico acima (Python, Javascript, Java, C++ e C) para a exibição de uma simples mensagem para o usuário escrita “Meu nome é Higor!”.

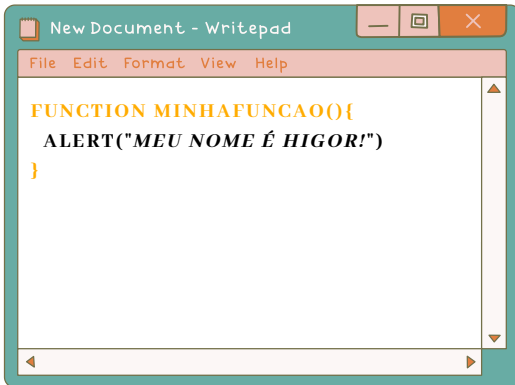
## • PYTHON



A screenshot of a Notepad window titled "New Document - Writepad". The window has a menu bar with "File", "Edit", "Format", "View", and "Help". The text area contains the following Python code:

```
PRINT("MEU NOME É HIGOR!")
```

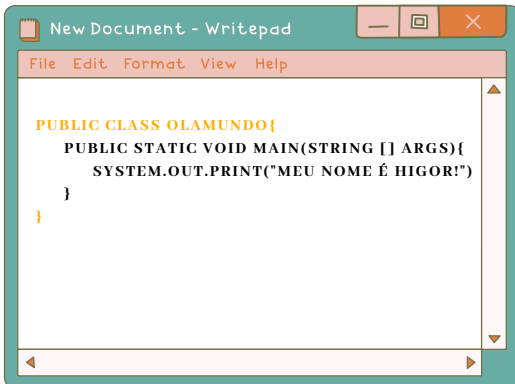
## • JAVASCRIPT



A screenshot of a Notepad window titled "New Document - Writepad". The window has a menu bar with "File", "Edit", "Format", "View", and "Help". The text area contains the following JavaScript code:

```
FUNCTION MINHAFUNCAO(){  
  ALERT("MEU NOME É HIGOR!")  
}
```

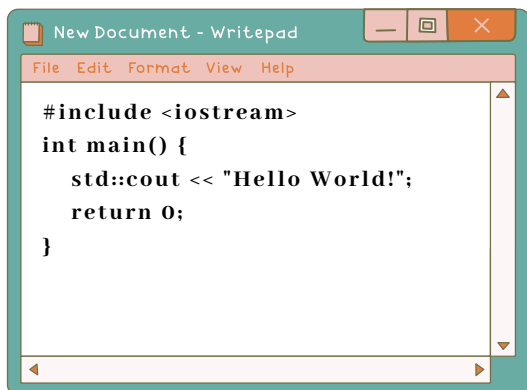
## • JAVA



A screenshot of a Notepad window titled "New Document - Writepad". The window has a menu bar with "File", "Edit", "Format", "View", and "Help". The text area contains the following Java code:

```
PUBLIC CLASS OLAMUNDO{  
  PUBLIC STATIC VOID MAIN(STRING [] ARGS){  
    SYSTEM.OUT.PRINT("MEU NOME É HIGOR!")  
  }  
}
```

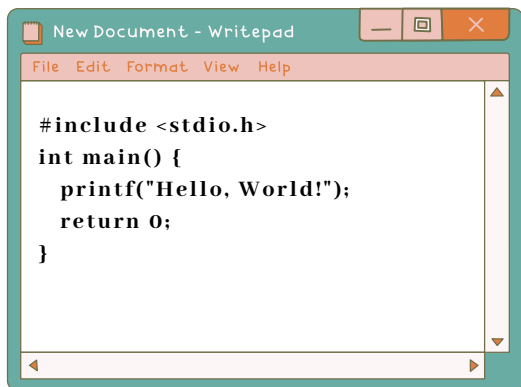
## • C++



A screenshot of a Notepad window titled "New Document - Writepad". The window has a menu bar with "File", "Edit", "Format", "View", and "Help". The text area contains the following C++ code:

```
#include <iostream>
int main() {
    std::cout << "Hello World!";
    return 0;
}
```

## • C



A screenshot of a Notepad window titled "New Document - Writepad". The window has a menu bar with "File", "Edit", "Format", "View", and "Help". The text area contains the following C code:

```
#include <stdio.h>
int main() {
    printf("Hello, World!");
    return 0;
}
```

**Bom, acho que deu para te convencer que Python tem várias vantagens. O quê ? Não está convencido ? Então talvez isto te convença!**

Aqui estão algumas das empresas que utilizam e investem pesado no Python:



<https://www.topdevelopers.co/blog/10-reasons-to-choose-python-web-development-project/>

Bom, acho que agora podemos continuar, certo ?

# VAMOS COM CALMA!

## VERSÕES DO PYTHON

Existem várias versões do Python, veremos isto próximo capítulo onde faremos a instalação do interpretador desta linguagem em nossa máquina.

A Versão que utilizaremos neste vídeo, e inclusive a mais atual é a Python 3.

Segue abaixo algumas das versões do Python.

| Release version               | Release date   |
|-------------------------------|----------------|
| <a href="#">Python 2.7.17</a> | Oct. 19, 2019  |
| <a href="#">Python 3.7.5</a>  | Oct. 15, 2019  |
| <a href="#">Python 3.8.0</a>  | Oct. 14, 2019  |
| <a href="#">Python 3.7.4</a>  | July 8, 2019   |
| <a href="#">Python 3.6.9</a>  | July 2, 2019   |
| <a href="#">Python 3.7.3</a>  | March 25, 2019 |
| <a href="#">Python 3.4.10</a> | March 18, 2019 |



## **POR QUÊ EXISTEM TANTAS VERSÕES DO PYTHON ?**

Imagine que você compre um celular de última geração como um Iphone, por exemplo, após alguns meses as tecnologias mudam, falhas de segurança são encontradas, correções de performance e atualizações de softwares são feitas, certo?

Não seria diferente para o Python, que possui em sua versão mais recente as correções das anteriores, então é recomendado que sempre seja utilizado a versão mais atual, que até o momento de escrita deste livro é a Python 3.

## O QUE PODEMOS FAZER PYTHON ?

Esta pergunta é interessante e a resposta é simples:

**VOCÊ PODE FAZER TUDO O QUE VOCÊ QUISER.**

Com Python, podemos automatizar coisas custosas, agilizar tarefas repetidas criarmos um site, investir em ações, criar jogos, criar um e-commerce, criar um sistema de gestão empresarial, criar bot(robôs) para o Instagram, Twitter, Facebook e etc.

Por ser uma linguagem robusta, também você consegue fazer projetos que estão em tendência como:

Machine Learning, Data Science, Deep Learning, Inteligência Artificial e Internet of Things.

Além de estar muito presente no mercado de Segurança da Informação.

Vai por mim, esses últimos tópicos são os principais temas que acontecerão no futuro(e quando eu digo futuro é daqui 2 anos), acredite. Nos Estados Unidos e em grandes centros que representam o "Vale do Silício" não se fala outra coisa a não ser de Python e o poder que esta linguagem possui.

# **PYTHON, O INVESTIMENTO DE CURTO PRAZO**

Em minha análise crítica sobre qual vai ser o percurso da tecnologia no Brasil nos próximos 3~5 anos, concluí que ainda somos bebês perto da China e EUA quando se trata de inovação tecnológica.

O Assunto mais procurado lá fora é: Machine Learning, Data Science, Deep Learning, Inteligência Artificial.

Enquanto no Brasil ainda temos pouca procura neste nicho, pois como somos um país emergente, temos em média 5 anos de atraso de informação e mercado em relação às grandes potências.

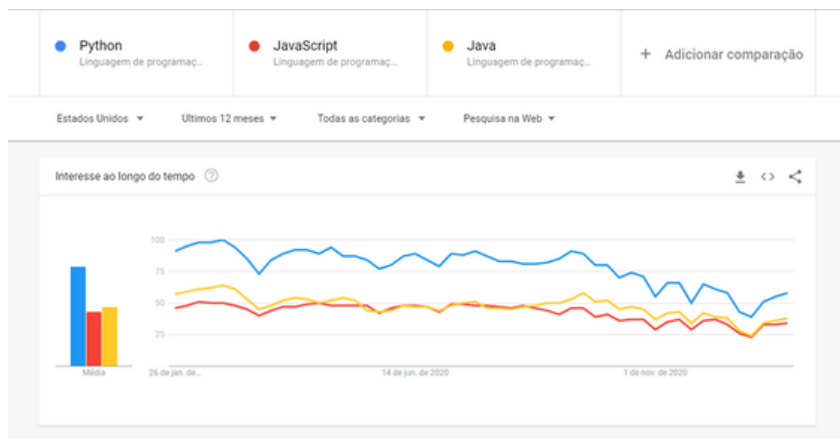
Nós já vimos que quem domina esses assuntos de inovação tecnológica e aprendizado de máquina é o Python, certo ? Também já vimos que é a linguagem mais procurada no Google, certo ?

Vamos fazer um breve comparativo com as principais linguagens buscadas no Google entre Brasil vs EUA.

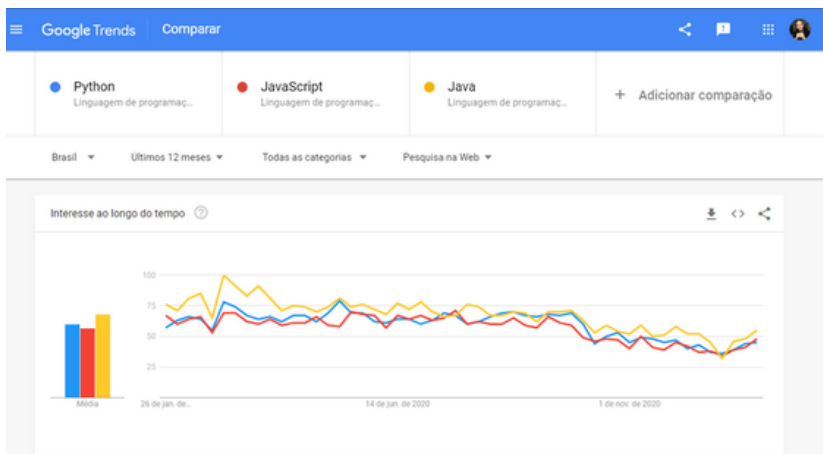
# BRASIL VS EUA



## Nos EUA:



## No Brasil:



# BRASIL VS EUA

Ainda estamos em fase de adaptação com as novas tecnologias que estão entrando no mercado e até isto virar algo sólido e direto no Brasil, dura um tempo.

Mas para os profissionais que desejam entrar nesta área HOJE, tem uma grande oportunidade de investimento no longo prazo.

## TÁ BOM, MAS E HOJE? EXISTE MERCADO ? QUEM VAI PAGAR MINHAS CONTAS ?

**Calma!**

Existem muitas vagas de Python no mercado brasileiro SIM.

Olha quantas empregos abertos no LinkedIn:

[br.linkedin.com > jobs > python-vagas](https://br.linkedin.com/jobs/python-vagas) ▾ [Translate this page](#)

**4.000+ vagas de Python em: Brasil (241 nova(s)) - LinkedIn**

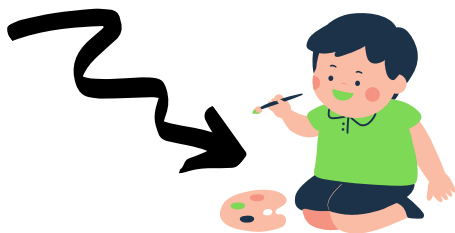
VAGA: DESENVOLVEDOR PYTHON + DJANGO. Descrição: Python, Django, Inglês fluente.

Área: Engenharia de Software / Banco de dados / ... Há 2 dias ...

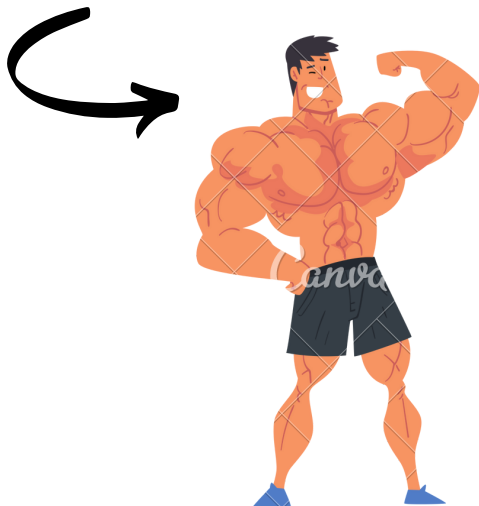
<https://br.linkedin.com/jobs/python-vagas?position=1&pageNum=0>

Por estes motivos mencionados acima, eu ousou dizer o "PODER DO PYTHON" ainda nem começou no Brasil e para aqueles que se atentarem e aprenderem esta linguagem ANTES, do que eu gosto de chamar de "A Grande Onda do Python", nadará de braçadas em um mar cheio de oportunidades e lógico, muita grana.

## PYTHON NO BRASIL



## PYTHON NOS EUA



# COMO INSTALAR PYTHON ?

Para todos as possíveis instalações para as diferentes plataformas abaixo mencionadas, você deve acessar o site:

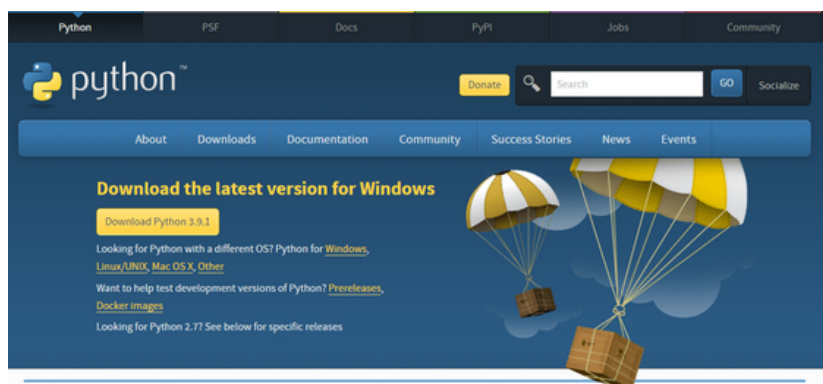


## Download Python

The official home of the Python  
Programming Language

[python.org](https://www.python.org)

<https://www.python.org/downloads/>



Basicamente Python é multi plataforma e para a grande maioria dos Sistemas Operacionais já vem como default, porém explicarei como fazer a instalação do interpretador Python no Windows, MacOS e Linux.

# INSTAÇÃO EM MÁQUINA WINDOWS

Após ser feito o download, você deve fazer a instalação:



Após seguir o famoso padrão Windows de “Next, Next and Next again” você pode verificar se o Python foi instalado corretamente em seu Sistema Operacional.

Para isso:

```
Prompt de Comando

PYTHONSTARTUP: file executed on interactive startup (no default)
PYTHONPATH : ';-separated list of directories prefixed to the
              default module search path. The result is sys.path.
PYTHONHOME : alternate <prefix> directory (or <prefix>;<exec-prefix>).
              The default module search path uses <prefix>\python[<major>]{<minor>}.
PYTHONPLATLIBDIR : override sys.platlibdir.
PYTHONCASEOK : ignore case in 'import' statements (Windows).
PYTHONUTF8 : if set to 1, enable the UTF-8 mode.
PYTHONIOENCODING: Encoding[errors] used for stdin/stdout/stderr.
PYTHONFAULTHANDLER: dump the Python traceback on fatal errors.
PYTHONHASHSEED: if this variable is set to 'random', a random value is used
                 to seed the hashes of str and bytes objects. It can also be set to an
                 integer in the range [0,4294967295] to get hash values with a
                 predictable seed.
PYTHONMALLOC: set the Python memory allocators and/or install debug hooks
               on Python memory allocators. Use PYTHONMALLOC=debug to install debug
               hooks.
PYTHONCOERCECLOCALE: if this variable is set to 0, it disables the locale
                      coercion behavior. Use PYTHONCOERCECLOCALE=warn to request display of
                      locale coercion and locale compatibility warnings on stderr.
PYTHONBREAKPOINT: if this variable is set to 0, it disables the default
                   debugger. It can be set to the callable of your debugger of choice.
PYTHONDEVMODE: enable the development mode.
PYTHONPYCACHEPREFIX: root directory for bytecode cache (.pyc) files.

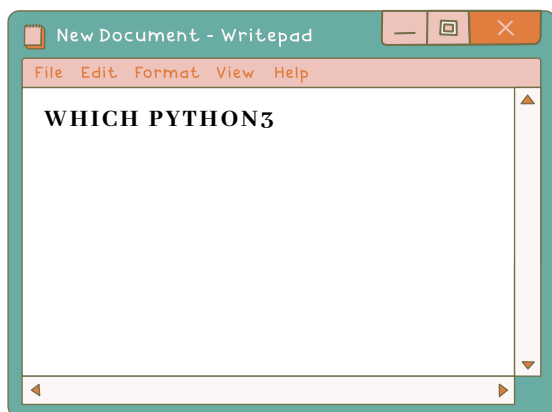
C:\Users\Higor>python --version
Python 3.9.1

C:\Users\Higor>
```

# INSTAÇÃO EM MÁQUINA

## MacOS

Na grande maioria das vezes o Python já vem instalado com este Sistema Operacional, para verificar isso tente digitar em seu terminal o seguinte comando:



Caso ocorra algum erro na execução do comando acima, aí você precisa fazer a instalação do XCode, que pode ser baixado na App Store, do pacote para desenvolvimento em linha de comando no macOS, command line tools e dos gerenciadores de pacotes pip e homebrew.

Para instalar o command line tools, digite em um terminal:

```
$ xcode-select --install
```

Para instalar o pip, digite em um terminal:

```
$ sudo easy_install pip
```

Para atualizar o pip, digite em um terminal:

```
$ sudo pip install --upgrade pip
```

Para instalar o homebrew, digite em um terminal:

```
$ ruby -e "$(curl -fsSL https://raw.githubusercontent.com/mxcl/homebrew/go)"
```

Para instalar o Python 2, digite em um terminal:

```
$ brew install python
```

Para instalar o Python 3, digite em um terminal:

```
$ brew install python3
```

<https://python.org.br/instalacao-mac/>

# INSTAÇÃO EM MÁQUINA

## LINUX

A maioria das versões do Linux já possuem Python como padrão de instalação, para verificar isso é só abrir o terminal e digitar "*python*" se não exibir nenhuma mensagem de erro o python já está instalado em sua máquina, mas se for exibido alguma mensagem de erro você deve seguir os passos:

### Instalação por Gerenciadores de Pacotes

Os gerenciadores de pacotes mais comuns são apt-get (Debian, Ubuntu) e yum (RedHat, CentOS). Caso sua distribuição utilize um gerenciador de pacotes diferente, acesse a [página de downloads do Python](#).

#### Apt-get

Para instalar o Python 3, digite em um terminal:

```
$ sudo apt-get install python3
```

(Opcional) Para instalar o gerenciador de pacotes pip, digite em um terminal:

```
$ sudo apt-get install python3-pip
```

#### Yum

Para instalar o Python 3, digite em um terminal:

```
$ sudo yum install python3
```

(Opcional) Para instalar o gerenciador de pacotes pip, digite em um terminal:

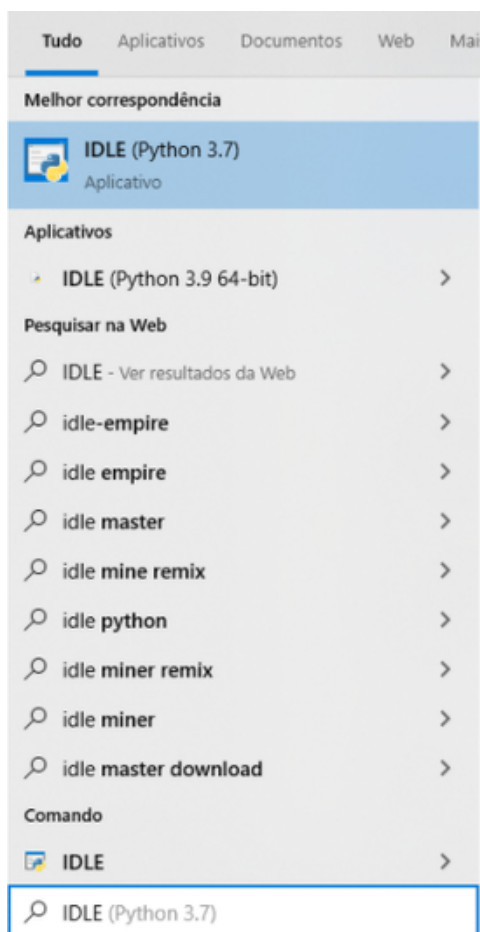
```
$ yum -y install python3-pip
```

<https://python.org.br/instalacao-linux/>

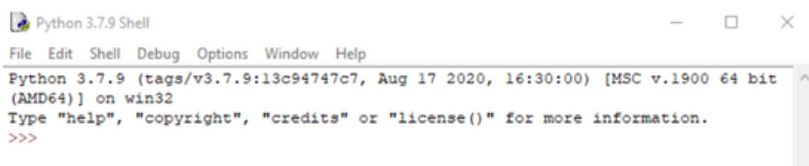
## CAPÍTULO 2: O QUE É IDLE ?

IDLE é um ambiente de desenvolvimento para o Python, ou seja, onde colocaremos nossos códigos.

Para encontrar o IDLE, se você está no Windows, procure no menu inicial por “IDLE”:



**Ao abrir você verá algo como:**

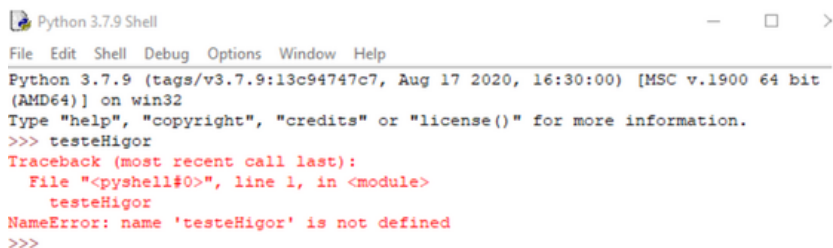


```
Python 3.7.9 Shell
File Edit Shell Debug Options Window Help
Python 3.7.9 (tags/v3.7.9:13c94747c7, Aug 17 2020, 16:30:00) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
```

basicamente tudo que digitarmos dentro do IDLE será interpretado como comandos Python.

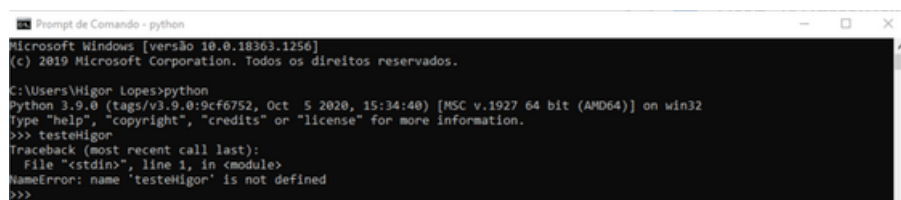
Ou seja, um interpretador, reconhece os comandos Python para traduzi-los a nível de máquina.

Tudo que vem depois do “>>>” espera receber algum comando Python para que seja feito o processamento, desta forma se digitarmos um comando “não Python” é exibida a mensagem:



```
Python 3.7.9 Shell
File Edit Shell Debug Options Window Help
Python 3.7.9 (tags/v3.7.9:13c94747c7, Aug 17 2020, 16:30:00) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> testeHigor
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    testeHigor
NameError: name 'testeHigor' is not defined
>>>
```

Caso você queira utilizar o CMD, ou seja, o prompt de comando no caso do Windows, sim ela mesmo “a telinha preta”, você pode ir no botão iniciar e digitar “CMD”, após abrir digite “python”, desta forma também será aberto o interpretador do Python em sua máquina e assim como no IDLE, sempre depois dos “>>>”, será esperado um comando Python para ser processado.



```
Prompt de Comando - python
Microsoft Windows [versão 10.0.18363.1256]
(c) 2019 Microsoft Corporation. Todos os direitos reservados.

C:\Users\Higor Lopes>python
Python 3.9.0 (tags/v3.9.0:9cf6752, Oct 5 2020, 15:34:40) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> testehigor
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'testehigor' is not defined
>>>
```

A nível de aprendizado, utilizaremos os dois modos neste livro, porém você pode utilizar qualquer um dos métodos exibidos acima.

Agora vamos nos preparar para escrever o nosso primeiro código em Python!

Diz a lenda que todas as vezes que precisamos aprender uma linguagem nova, o primeiro caminho a ser tomado é o “Hello World”, caso contrário não teremos sucesso na linguagem que estamos estudando...

Bom, eu nunca quis duvidar desta lenda urbana, então vamos lá!

Exibiremos uma mensagem de texto escrita "Hello World", para isso digitamos o comando:

```
Python 3.7.9 (tags/v3.7.9:13c94747c7, Aug 17 2020, 16:30:00) [MSC v.1900 64 bit  
(AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>> print("Hello World!")  
Hello World!
```

O comando "print" serve para exibir uma mensagem, ele é muito importante e utilizaremos ele a todo momento em nossa trajetória como desenvolvedor Python.

 Agora você pode se perguntar,  
*"Legal Higor, então eu acabei de fazer  
meu primeiro programa em Python?"*

Definitivamente, não!

Entenda que estamos mexendo apenas no INTERPRETADOR da linguagem Python, que como havíamos conversado, apenas ENTENDE e TRADUZ os comandos, funções e métodos que inserimos em nosso computador.

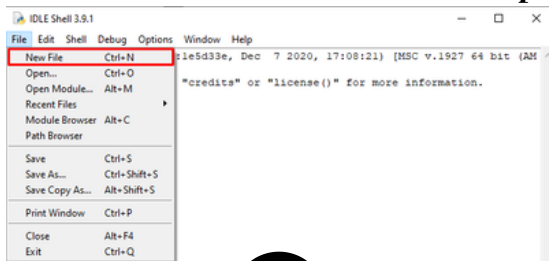
 *"Tá! Então, como que você pode  
conseguir criar um programa em  
Python?"*

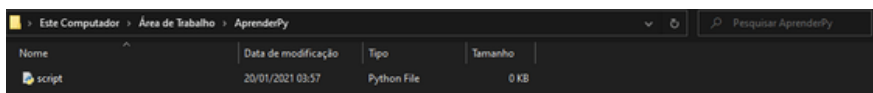
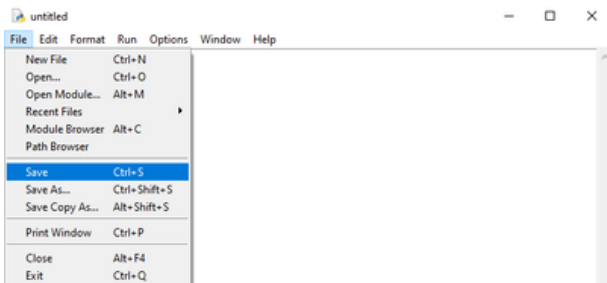
# MODOS DE CRIARMOS ARQUIVOS PYTHON.

Os arquivos em Python são referenciados pela extensão **.PY**, podemos criar programas Python das seguintes formas:

- Abrir um bloco de notas e salvar um arquivo vazio com a extensão **.py** ao invés de **.txt**
- A partir do próprio IDLE, usando as teclas **Ctrl + N** ou indo em File -> New File e salvando.
- Utilizar Editores de Texto mais profissionais como vsCode, Sublime Text, Notepad++ e salvar o arquivo como **.py**.
- Utilizar uma IDE específica para programação em Python como pyCharm.

Todos os métodos passados acima tem um único objetivo, criar um programa Python. Em nosso exemplo criaremos uma pasta na área de trabalho chamada **aprenderPy** e também criaremos um programa **Python** a partir do IDLE e salvaremos nesta pasta.

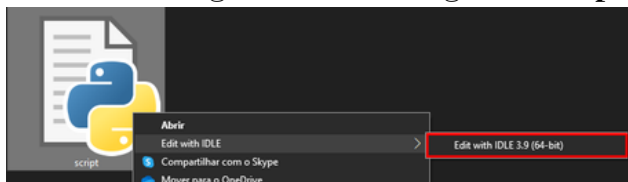




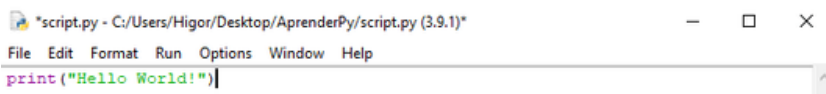
**Se caso você estiver usando Windows, verá que todo programa Python ficará com o ícone semelhante a este:**



**Podemos editar um arquivo .py pelo próprio IDLE seguindo os seguintes passos:**



**Agora sim podemos adicionar nosso código de impressão de mensagem igual já havíamos feito no interpretador, porém agora estamos fazendo em nosso programa**



**Seu programa está feito, mas como você vai conseguir executá-lo ?**

# EXECUTANDO ARQUIVOS PYTHON

Agora que o programa está pronto, o que nos resta é executá-lo, para isso podemos utilizar o CMD, mas antes você precisa ENTENDER um pouco sobre navegação entre pastas do seu Sistema Operacional:

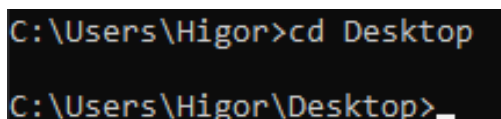


```
Prompt de Comando
Microsoft Windows [versão 10.0.18363.1316]
(c) 2019 Microsoft Corporation. Todos os direitos reservados.
C:\Users\Higor>
```

Para que seja possível a visualização e acesso ao arquivo `script.py` que acabou de ser criado nos passos anteriores, PRECISAMOS indicar, ou seja, estarmos na "mesma pasta" e em meu cenário eu estou na pasta:

`C:\Users\Higor>`

Se digitarmos o comando *DIR*, aparecerá as subpastas ou arquivos referentes a pasta `C:\Users\Higor>`, se você salvou seu programa Python na Área de Trabalho você precisa navegar até o "*Desktop*":



```
C:\Users\Higor>cd Desktop
C:\Users\Higor\Desktop>
```

Agora que estamos na Área de Trabalho, chamada Desktop no nosso CMD, podemos entrar em nossa pasta criada(**AprenderPy**):

```
C:\Users\Higor\Desktop>dir
O volume na unidade C não tem nome.
O Número de Série do Volume é 6415-F4E6

Pasta de C:\Users\Higor\Desktop

20/01/2021  09:20    <DIR>        .
20/01/2021  09:20    <DIR>        ..
20/01/2021  03:57    <DIR>        AprenderPy
20/01/2021  09:20      53.612 d448e55d-59b2-4c0f-b4bd-9eb988b4c527.jpg
19/01/2021  06:25      2.227 Discord.lnk
18/01/2021  11:37      2.474 Higor (Higor Granero) - Chrome.lnk
10/09/2020  17:57      2.356 Microsoft Teams.lnk
19/01/2021  18:00      113 script.py
18/01/2021  11:36      2.474 Seven (Sev7n Glass) - Chrome.lnk
18/01/2021  11:35    <DIR>        WSG
                6 arquivo(s)        63.256 bytes
                4 pasta(s)   112.001.277.952 bytes disponíveis

C:\Users\Higor\Desktop>cd AprenderPy
```

Ao entrarmos na pasta encontramos nosso programa Python *script.py*:

```
C:\Users\Higor\Desktop>cd AprenderPy
C:\Users\Higor\Desktop\AprenderPy>dir
O volume na unidade C não tem nome.
O Número de Série do Volume é 6415-F4E6

Pasta de C:\Users\Higor\Desktop\AprenderPy

20/01/2021  03:57    <DIR>        .
20/01/2021  03:57    <DIR>        ..
20/01/2021  04:04      23 script.py
                1 arquivo(s)        23 bytes
                2 pasta(s)   111.978.889.216 bytes disponíveis
```

Para executarmos nosso programa devemos:

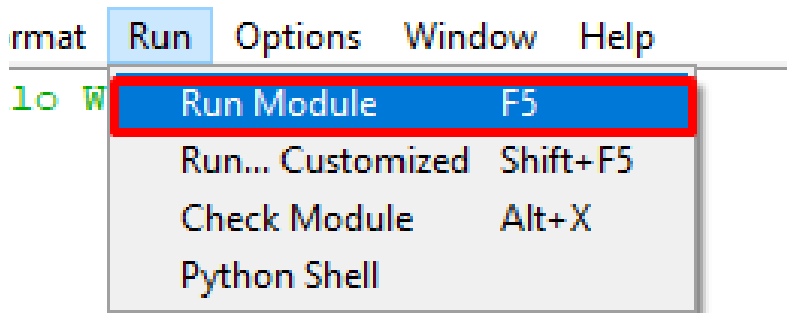
```
C:\Users\Higor\Desktop\AprenderPy>python script.py
```

Após executarmos nosso programa o resultado é:

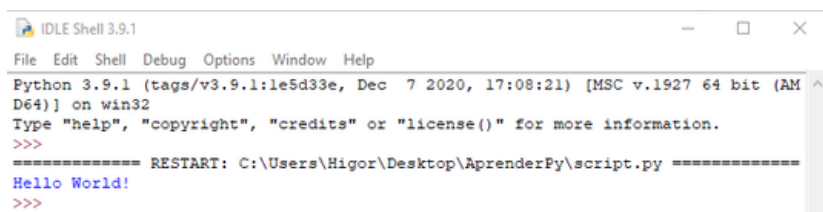
```
C:\Users\Higor\Desktop\AprenderPy>python script.py
Hello World!
```

Existe um modo mais fácil de executarmos um programa Python, pelo próprio IDLE.

C:\Users\Higor\Desktop\AprenderPy\script.py (3.



Neste caso o resultado é exibido da seguinte forma:



Eu prefiro utilizar o CMD, pois você aprende a estrutura básica de um S.O(Sistema Operacional), estruturas de diretórios, navegações entre diretórios, sem utilizar interface gráfica, e no Linux(O S.O queridinho de programadores) este conhecimento é OBRIGATÓRIO.

# COMENTÁRIOS EM PYTHON

Agora que temos nosso programa, vamos conhecer algo que é muito utilizado pelos programadores que são os COMENTÁRIOS, que servem para explicar um código para próximos desenvolvedores ou para você mesmo depois de um determinado tempo.

Basicamente um comentário é um bloco que não é processado pelo Python, ou seja, tudo que é referenciado como um comentário é desconsiderado pelo interpretador, por este motivo nós somos livres para escrevermos qualquer coisa, DESDE QUE siga as seguintes formas de comentário.

No Python existem dois tipos de comentários:

**Comentário em Massa & Comentário por Linha.**

**Comentário por Linha:**

```
#Isto é um comentário - by: Higor :)  
#Me siga no Instagram - @higorgranero
```

Desconsidera/Comenta Tudo que está depois do sinal (#) na mesma linha.

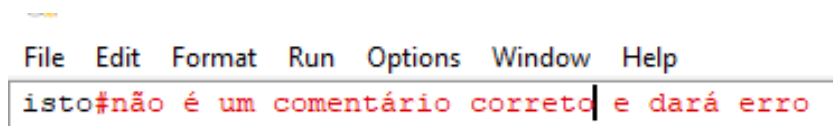
## Comentário em Massa:

```
'''  
Isto é um comentário - by: Higor :)  
Me siga no Instagram - @higorgranero  
Me acompanhe no Youtube - Higor Granero  
'''
```

Desconsidera/Comenta tudo que está entre os sinais( `` )

## ATENÇÃO:

Os comentários só farão efeito para o que estiver ENTRE sinais `` ou #



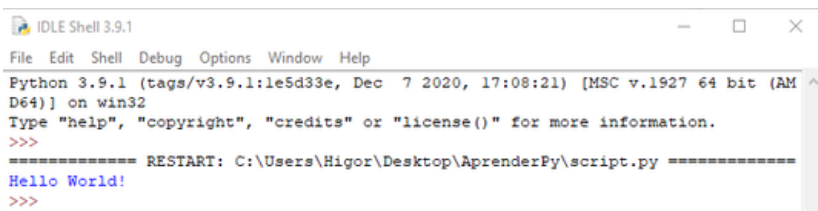
The screenshot shows a window with a menu bar (File, Edit, Format, Run, Options, Window, Help) and a text area. The text in the area is "isto#não é um comentário correto" followed by a cursor. A red error message "e dará erro" is displayed at the end of the line.

Como dito, tudo que está **ENTRE** ou **DEPOIS** dos sinais mencionados acima o **Python** entende que não deve ser processado, pois trata-se de um **COMENTÁRIO**, por este motivo podemos escrever **QUALQUER** coisa.

Vamos adicionar um comentário em nosso programa *script.py* criado.

```
#Comando para exibir mensagem em Python  
#Meu primeiro Hello World :)  
print("Hello World!")
```

Se executarmos novamente o programa dará o mesmo resultado, pois o comentário não interfere no programa a nível de processamento.



```
IDLE Shell 3.9.1
File Edit Shell Debug Options Window Help
Python 3.9.1 (tags/v3.9.1:1e5d33e, Dec 7 2020, 17:08:21) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\Higor\Desktop\AprenderPy\script.py =====
Hello World!
>>>
```

Neste exemplo o comentário não foi tão útil, mas imagina em projetos gigantescos que passam pelas "mãos" de vários outros programadores que podem ou não serem experientes ?

Lembro-me de alguns projetos, no início da minha carreira, que eu não entendia a lógica que o programador quis usar para desenvolver um processo X e ficava dias e dias debugando, sendo que em um único comentário poderia tirar minha dúvida.

Comentários é a forma mais eficaz e eficiente de dizer:

"Eu te amo e quero o teu bem" para um programador.

Então por favor!

**COMENTE SEMPRE O SEU CÓDIGO!**

# CAPÍTULO 3:

## Variáveis e Operadores

Aqui falaremos de uma das coisas mais importantes em qualquer linguagem de programação:

### Variáveis e Operadores

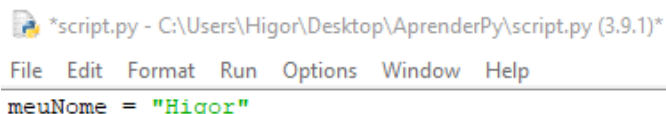
O que são Variáveis ?

Como o próprio nome diz, uma variável pode acumular nenhum, um ou vários conteúdos ao decorrer do processo de seu programa.

Não podemos esquecer que a variável só existe em tempo de processamento, ou seja, enquanto seu programa está rodando, caso você feche o programa e re-executa, a variável não obrigatoriamente terá o mesmo valor do primeiro processo.

Dessa forma as variáveis são usadas apenas para armazenar temporariamente um valor que logo será apagado.

Em Python declaramos variáveis da seguinte forma:



The image shows a screenshot of a Python script editor. At the top, the title bar reads: `*script.py - C:\Users\Higor\Desktop\AprenderPy\script.py (3.9.1)*`. Below the title bar is a menu bar with the following items: `File`, `Edit`, `Format`, `Run`, `Options`, `Window`, and `Help`. The main text area contains the line of code: `meuNome = "Higor"`, where the string `"Higor"` is highlighted in green.

Neste momento, toda vez que o programa ler esta variável(**meuNome**) ele armazenará temporariamente na memória o conteúdo de Nome da Variável(**meuNome**) e Conteúdo da Variável(**Higor**).

Se caso precisarmos exibir o conteúdo da variável **meuNome** para o usuário, assim como fizemos anteriormente na página X, podemos fazer o seguinte:

```
print(meuNome)
```

 \*script.py - C:\Users\Higor\Desktop\AprenderPy\script.py (3.9.1)\*

File Edit Format Run Options Window Help

```
meuNome = "Higor"  
print(meuNome)
```

Desta forma podemos criar várias variáveis, uma para cada objetivo, exemplo:

```
meuNome = "Higor"
```

```
minhaIdade = 21
```

```
meuSobrenome = "Granero"
```

Para agilizar o Python também nos permite criarmos VÁRIAS VARIÁVEIS tudo em uma única linha:

```
meuNome, minhaIdade, meuSobrenome = "Higor", 21, "Granero"
```

# Nomenclatura de Variáveis

As Variáveis em **Python** podem conter apenas Letras, maiúsculas e minúsculas, número (com exceção na primeira posição no nome da variável) e underline (\_).

Além disso, não podemos utilizar algumas palavras que são restritas do **Python** como:

**if, input, while, for e etc...**

(depois falaremos mais deles)

Por último o **Python** é case sensitive, ou seja, a variável “**meuNome**” é diferente de “**meunome**”, consequentemente ambas terão espaços diferentes de armazenamento na memória temporária e cada uma armazenará um valor.

# Atribuição

Ao contrário no nosso entendimento em matemática, em **Python** e em outras linguagens o sinal de “=” é referente a atribuição, ou seja, suponhamos que temos uma variável **nomeChachorro**, para atribuímos um valor a esta variável, colocamos o sinal de atribuição(sinal de igual) e o conteúdo que gostaríamos de armazenar nesta variável.

Exemplo:

**nomeChachorro** = "Pitbull"



Nome Variável



Conteúdo Variável

Isso é bem básico, mas se este conceito não estiver firmado em sua mente você pode falhar em entender um algoritmo simples como este

**x** = 1

**y** = 2

**x** = **y**

**print**("x = ", **x**)

**print**("y = ", **y**)

O que será exibido no valor de Y e X ?

Como mencionamos o sinal de “=” é relacionado a atribuição, então ambas as variáveis iniciaram com os valores, X com 1 e Y com 2, porém na linha 3 X atribuiu o valor da variável Y, desta forma o conteúdo exibido deverá ser:

**X** = 2

**Y** = 2

## Operações Básicas

É claro que em Python podemos fazer operações matemáticas, estas operações básicas consistem em adições, subtrações, divisões, multiplicações.

Exemplo:

Suponhamos que temos duas variáveis A e B, podemos criá-las da seguinte forma, igual vimos anteriormente:

**Declaração de variável:**

**A, B** = 5, 10

Como podemos fazer operações matemáticas com essas variáveis ?

**Adição:**

$$A + B = 15$$

**Subtração:**

$$A - B = 5$$

**Multiplicação:**

$$A * B = 50$$

**Divisão:**

$$A / B = 2$$

**Módulos:**

$$A \% B = 0$$



(retorna o resto da divisão, muito útil para saber se o número é ímpar ou par, se o resto do número for 0, é par se for ímpar é 1)

**Expoente:**

$$B ** A = 100000$$



(10 com expoente de 5)

**Divisão exata:**

$$9 / 4 = 2$$

## Mais Algumas Operações

Além das operações básicas, no Python (e em outras linguagens), possuímos algumas operações juntas com atribuições como

$+=$ ,  $-=$  e  $*=$ .

Suponhamos que declaramos novamente duas variáveis:

$X, Y = 6, 3$

Se caso quiséssemos fazer uma adição da variável  $X$  com o número inteiro 4, poderíamos fazer:

$X = X + 4$ , correto ?

Por baixo dos panos temos:

$6 = 6 + 4$ , neste caso como o sinal de “=” é **ATRIBUIÇÃO** o valor da variável  $X$  mudará para 10.

Mas ao invés de fazermos desta forma mais longa ( $X = X + 4$ ), podemos utilizar a seguinte Operação  $X += 4$ .

Ambas possuem o MESMO significado, da mesma forma se quiséssemos realizar a subtração do número 4 na variável  $X$ , poderíamos fazer da seguinte forma:

$X = X - 4$

ou

$X -= 4$

Em resumo, o que estes sinais querem dizer é:

$$X = 20$$

Nome  
Variável

←...X

+ = 10

...→

Valor a Ser  
atribuído a  
Variável X



ATRIBUA o  
valor de X + 10  
na Variável X.

OPERAÇÃO que  
você deseja  
fazer com X,  
neste caso,  
será SOMADO o  
valor de X com  
o valor de 10

## CAPÍTULO 4:

# Tipos de Dados em Python

Tipos de dados básicos suportados pelo Python são Inteiro, Float e String.

Falaremos um pouco de cada um destes neste capítulo e depois discutiremos sobre tipos complexos de dados no capítulo 5.

Tipos de dados:

### Inteiro:

São números decimais, ou seja, números negativos e positivos.

Exemplo:

*numeroCasa* = 123

### Float:

Fazem parte dos números decimais porém são números quebrados.

Exemplo:

*meuPeso* = 67.2

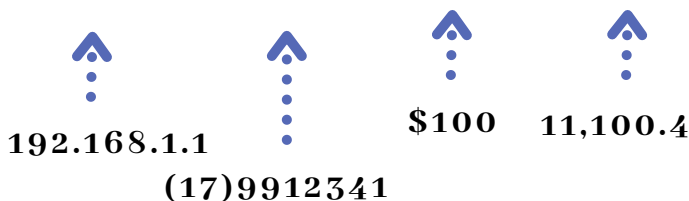
# O QUE SÃO E O QUE NÃO SÃO NÚMEROS.

Como vimos existem dois tipos de números em Python, Inteiros e flutuantes(float). Abaixo coloquei mais exemplos daquilo que é um número e o que não é.

## NÚMERO

  
1 1.1 12345.76 -10 .99

## NÃO NÚMERO

  
192.168.1.1 (17)9912341 \$100 11,100.4

## String:

Strings estão relacionadas aos textos, para declarar uma string você deve colocar aspas simples ou duplas.

Exemplo:

```
meuNome = "Higor"
```

```
meuSobrenome = 'Granero'
```

No Python existem várias funções e módulos para manipulação de String, uma função nada mais é que um bloco de código que é reutilizado, falaremos mais dele ao longo da leitura deste livro.

Um exemplo de uma função é a **UPPER()**, ela serve para colocar todas as letras em maiúsculo.

Se fizermos `meuNome.upper()` o retorno será **"HIGOR"**.

No Próximo tópico você possui uma lista completa de funções manipuladoras de string.

# Manipuladores de Strings

## Lista com as principais funções manipuladoras de String no Python:

- Captar o Tamanho de uma String:

- `len(nomeVariavel)`

- Exemplo:

```
>>> nomeVariavel = "Higor"
>>> len(nomeVariavel)
```

- Fragmentar String separada por palavras:

- `nomeVariavel.split()`

- Exemplo:

```
>>> nomeVariavel = "Qualquer Frase aqui!"
>>> nomeVariavel.split()
['Qualquer', 'Frase', 'aqui!']
```

- Contar número de vezes que uma letra está em uma palavra:

- `nomeVariavel.count()`

- Exemplo:

```
>>> nomeVariavel = "Qualquer Frase aqui!"
>>> nomeVariavel.count('a')
3
```

- Colocar todas as letras de uma string minúscula:

- `nomeVariavel.lower()`

- Exemplo:

```
>>> nomeVariavel = "Qualquer Frase aqui!"
>>> nomeVariavel.lower()
'qualquer frase aqui!'
```

- Colocar a primeira letra de cada palavra maiúscula:

- `nomeVariavel.title()`

- Exemplo:

```
>>> nomeVariavel = "Qualquer Frase aqui!"
>>> nomeVariavel.title()
'Qualquer Frase Aqui!'
```

- Verificando se a string possui apenas letras:

- `nomeVariavel.isalpha()`

- Exemplo:

```
>>> nomeVariavel = "Higor"
>>> nomeVariavel.isalpha()
True
```

- Verificando se a string possui apenas números:

- `nomeVariavel.isdigit()`

- Exemplo:

```
>>> nomeVariavel = "123"
>>> nomeVariavel.isdigit()
True
```

- Verificando se a string possui apenas espaços em branco:

- `nomeVariavel.isspace()`

- Exemplo:

```
>>> nomeVariavel = "   "
>>> nomeVariavel.isspace()
True
```

- Adicionando espaços à direita de uma string:

- `nomeVariavel.ljust(X)`

- Exemplo:

```
>>> nomeVariavel = "123"
>>> nomeVariavel.ljust(10)
'123          '
```

- Adicionando espaços à esquerda de uma string:

- `nomeVariavel.rjust(Y)`

- Exemplo:

```
>>> nomeVariavel = "123"
>>> nomeVariavel.rjust(10)
'          123'
```



*"Mas Higor. o que são funções ?"*

Acalme-se, falaremos mais sobre funções em um capítulo específico logo mais abaixo neste livro.

## O TIPO BOLEANO

Além de números(int & float) e String, existe um terceiro tipo de dados em Python que não é exatamente um número ou uma string, chamado de booleano (em homenagem a um matemático chamado George Boole), e pode ser um de dois valores: **True** ou **False**.

Apesar de parecer "limitado" este tipo de dado é primordial para realizar tarefas a nível computacional, como veremos mais a frente, a lógica booleana está presente em validações condicionais, comparações, laços de repetição e etc...

Podemos adicionar o valor de True or False em variáveis criadas, como:

```
>>> a = True
>>> b = False
```

# EXIBINDO MENSAGENS CONCATENADAS EM PYTHON

## DESAFIO:

Faça um programa com 3 Variáveis:

**nome**, **idade** e **peso**.

Após a criação e atribuição de cada variável mostre na tela com o comando **print()** a seguinte mensagem:

"Meu nome é [Conteúdo da Variável nome]  
Minha idade é [Conteúdo da Variável idade] e meu peso é [Conteúdo da Variável peso]"

## Resolução do Desafio acima:

```
>>> nome = "Higor"
>>> idade = 27
>>> peso = 67.5
>>> print("Meu nome é " + nome + " minha idade é " + idade + " meu peso é " + peso)
Traceback (most recent call last):
  File "<pyshe11#3>", line 1, in <module>
    print("Meu nome é " + nome + " minha idade é " + idade + " meu peso é " + peso)
TypeError: can only concatenate str (not "int") to str
```

Como podemos ver, ocorreu um erro no interpretador, isto ocorre pois os dois últimos dados não são Strings, sendo assim podemos corrigir este erro com duas formas.

## Primeira Forma

(A mais utilizada e Recomendada).

Utilizamos o operador “%” para simbolizar que naquela parte da mensagem específica será inserida uma variável e a mesma terá tipo:

- %s (String),
- %d(Decimal) ou
- %f(Float).

Temos que referenciar qual ou quais variáveis ocuparão o espaço na string, como mostra a imagem abaixo:

```
>>> print("Meu nome é %s minha idade é %d meu peso é %f" %(nome,idade,peso))
Meu nome é Higor minha idade é 27 meu peso é 67.500000
>>>
```

## Segunda Forma:

Para corrigirmos o erro exibido na página anterior, podemos converter os dados de float ou int para string da seguinte forma:

```
>>> print("Meu nome é " + nome + " minha idade é " + str(idade) + " meu peso é "
+ str(peso))
Meu nome é Higor minha idade é 27 meu peso é 67.5
>>>
```

Falaremos de conversores no próximo capítulo.

# CONVERSORES

Como acabamos de ver, muitas das vezes precisamos converter um tipo de dados para outro tipo, exemplo, por algum motivo converter um Número Inteiro para um Texto String.

No Python, temos 3 funções que permitem fazer isto.

- `int()`
- `float()`
- `str()`

`int()` recebe uma string ou float e converte para número inteiro, exemplo:

```
>>> exemplo = "30"    >>> int(peso)
>>> int(exemplo)      67
30
```



STRING PARA INT



FLOAT PARA INT

## ATENÇÃO:

Não podemos converter uma variável com o conteúdo "Higor" para Inteiro, pois este conteúdo não é conversível

## CAPÍTULO 5:

# Listas em Python

Como o próprio nome diz lista é um conjunto de elementos juntos, por exemplo: imagina que você vai no mercado, e para lembrar de todas as coisas que precisa comprar, você decide anotar em uma folha de caderno cada item.

Você começa escrevendo a partir da primeira linha da folha, né ?



Suponhamos que em sua lista você insere:

- Ovos
- Arroz
- Massa
- Detergente
- Açúcar

Para fazermos isto no Python ao invés de uma folha de papel, seria:

```
>>> lista = ["ovos", "arroz", "massa", "detergente", "açúcar"]
>>>
```

Podemos referenciar posições para busca de um elemento dentro de uma lista, voltando no exemplo do mercado, se a prateleira de açúcar(último item) está perto da prateleira de ovos(primeiro item), podemos pular direto para a última posição e buscarmos o açúcar, correto ?

Da mesma forma ocorre com Python, para buscar um elemento em uma lista, usamos os colchetes [ ] passamos a posição que queremos buscar, que sempre é um número inteiro.

```
>>> lista[1]
'arroz'
>>>
```

Algo interessante a ser notado, é que passamos a posição "1" e ao invés dele trazer a "primeira" posição da lista, ele trouxe a "segunda", isto ocorre pois o Python e também outras linguagens consideram como primeira posição da lista o Número 0.

```
>>> lista[0]
'ovos'
>>>
```

Também no Python, podemos referenciar um número inteiro negativo para nossa lista, desta forma ela retorna para o final da lista.

Exemplo:

```
>>> lista[-1]
'açúcar'
>>>
```

Ou seja, quando colocamos um número negativo, a lista é lida de trás para frente.

Obviamente podemos ter vários dados diferentes dentro de uma mesma lista:

```
>>> print("Meu nome é %s %s , tenho %d anos e peso %f" %(pessoa[0], pessoa[1], pessoa[2], pessoa[3]))
Meu nome é Higor Lopes , tenho 21 anos e peso 65.500000
>>>
```

Imagina que você não fez uma lista de compras, e prefere chegar no mercado e ir montando sua lista de acordo com o que você vai tendo em mente ao ver os produtos nas prateleiras.

Neste caso, até o momento que você não decidiu o que comprar, você deixará sua lista(papel) em branco, certo ?

Para declarar uma lista em branco no Python, é só não inserir nenhum item, apenas com [ ] vazio, da seguinte forma:

```
>>> listaMercado = []  
>>>
```

Ao chegar no mercado, você decide fazer uma macarronada, então você precisa inserir itens na sua lista que até então estava vazia, o primeiro item que você irá adicionar será o "macarrão", para isso usamos o método `append()`:

```
>>> listaMercado.append("macarrão")  
>>> listaMercado  
['macarrão']  
>>>
```

O método `append()` SEMPRE insere na última posição de nossa lista, ou seja a posição 0.

Você percebe que não pode fazer macarronada somente com macarrão, então precisa colocar mais itens em sua lista, no Python podemos inserir item a item com o método `append()` igual vimos, ou podemos utilizar o método `extend`, que insere vários itens de uma vez em sua lista.

```
>>> listaMercado.append("macarrão")  
>>> listaMercado  
['macarrão']  
>>> listaMercado.extend(["molho de tomate", "tempero", "calabresa"])  
>>> listaMercado  
['macarrão', 'molho de tomate', 'tempero', 'calabresa']  
>>>
```

Bom, agora você já tem sua lista!

Mas você acaba de lembrar que o tempero você já tem na sua casa, não precisa comprar outro, então você decide deletar este item.

Para deletarmos um item no Python utilizamos o comando `del` passando o nome da lista e o item da lista que queremos deletar:

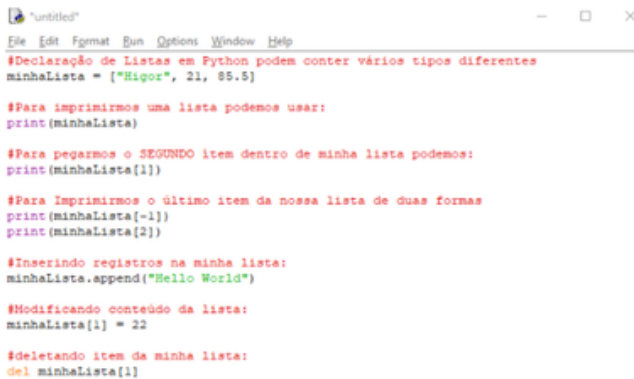
```
>>> listaMercado
['macarrão', 'molho de tomate', 'tempero', 'calabresa']
>>> del listaMercado[2]
>>> listaMercado
['macarrão', 'molho de tomate', 'calabresa']
>>>
```

Você também percebe que prefere um macarrão branco, e ao invés de molho de tomate você gostaria de trocar para “creme de leite”.

Para modificarmos um item de nossa lista no Python, podemos fazer da seguinte forma.

```
>>> listaMercado
['macarrão', 'molho de tomate', 'calabresa']
>>> listaMercado[1] = "creme de leite"
>>> listaMercado
['macarrão', 'creme de leite', 'calabresa']
>>>
```

## Recapitulando:



```
"untitled"
File Edit Format Run Options Window Help
#Declaração de Listas em Python podem conter vários tipos diferentes
minhaLista = ["Rigor", 21, 85.5]

#Para imprimirmos uma lista podemos usar:
print(minhaLista)

#Para pegarmos o SEGUNDO item dentro de minha lista podemos:
print(minhaLista[1])

#Para Imprimirmos o último item da nossa lista de duas formas
print(minhaLista[-1])
print(minhaLista[2])

#Inserindo registros na minha lista:
minhaLista.append("Hello World")

#Modificando conteúdo da lista:
minhaLista[1] = 22

#deletando item da minha lista:
del minhaLista[1]
```

# Tuplas

Tuplas são quase idênticas as listas, com a única diferença que não podem ser modificadas, o valor inicial de uma tupla ficará até o final do programa, seria basicamente um valor constante.

Para declararmos as tuplas no Python ao invés de colchetes [ ] (Usado nas listas), colocamos parênteses ( ).

```
>>> nomeTupla = ("Segunda-Feira", "Terça-Feira", "Quarta-Feira", "Quinta-Feira", "Sexta-Feira", "Sábado", "Domingo")
>>> nomeTupla
('Segunda-Feira', 'Terça-Feira', 'Quarta-Feira', 'Quinta-Feira', 'Sexta-Feira', 'Sábado', 'Domingo')
>>>
```

Podemos “navegar” nas tuplas, assim como fazemos com as listas, passando o índice(posição) do item que desejamos buscar.

```
('Segunda-Feira', 'Terça-Feira', 'Quarta-Feira', 'Quinta-Feira', 'Sexta-Feira', 'Sábado', 'Domingo')
>>> nomeTupla[1]
'Terça-Feira'
>>>
```

# Dicionários

Dicionários são dados em pares, imagine que você seja professor e precisa armazenar o nome e a nota de 3 alunos que participaram de um exame de recuperação, para isso podemos usar o dicionário:

```
>>> meuDicionario = {"Higor":2 , "Samara":10, "Maria":10}  
>>> meuDicionario  
{'Higor': 2, 'Samara': 10, 'Maria': 10}  
>>>
```

é possível usar o método *dict* para declararmos um dicionário:

```
>>> nomeNota = dict(Higor = 2, Samara = 10, Maria = 10)  
>>> nomeNota  
{'Higor': 2, 'Samara': 10, 'Maria': 10}  
>>>
```

Você pode se perguntar:

*"Por qual motivo usamos COLCHETES no primeiro exemplo e PARENTESSES no segundo ?"*

Isto ocorre pois *dict* é um método e método usado em Python é obrigatoriamente chamado com parênteses, por este motivo, ao invés de *{ }* usarmos *( )* no segundo exemplo.

Falaremos disso mais tarde, mas em resumo um método é similar a uma função, onde são passados parâmetros, que no caso do *DICT* , é feito internamente um dicionário.

Para modificar um item do dicionário, faremos da seguinte forma:

```
>>> nomeNota["Samara"] = 5
>>> nomeNota
{'Higor': 2, 'Samara': 5, 'Maria': 10,
>>>
```

Repare que o que vem antes do ":" é como se fosse o índice da lista que estudamos, ou seja, a posição e após o ":" é o conteúdo.

O que fizemos acima nada mais foi do que procurar no dicionário o índice referente a "Samara" e substituir o valor de 10 para 5.

Algo interessante é que se passarmos um índice inexistente, este conteúdo é inserido na lista

```
>>> nomeNota["Samra"] = 5
>>> nomeNota
{'Higor': 2, 'Samara': 5, 'Maria': 10, 'Samra': 5}
>>>
```

Para removermos um item de nosso dicionário, é idêntica a remoção da lista:

```
>>> del nomeNota["Higor"]
>>> nomeNota
{'Samara': 5, 'Maria': 10, 'Samra': 5}
>>>
```

## CAPÍTULO 6:

# PROGRAMA INTERATIVO COM PYTHON

Na maioria das vezes, precisamos e necessitamos que o usuário insira seus dados para que o programa seja processado, por isso existem os programas interativos.

Imagine que você esteja desenvolvendo um programa que fará o cálculo da média dos alunos, então para cada aluno que executar seu programa as notas serão obviamente diferentes, neste caso precisamos de que o usuário insira os valores, no Python usamos o comando *input()* para isso.

Exemplo do comando *Input()*:

```
>>> nomeUsuario = input("Digite seu nome: ")
Digite seu nome: Higor
```

Depois que o usuário digitar o seu nome, o conteúdo digitado é atribuído para a variável *nomeUsuario*.

Podemos exibir o conteúdo desta variável com o `print()`, igual já vimos.

```
>>> print(nomeUsuario)
Higor
>>>
```

# ASPAS TRIPLAS

Se precisarmos exibir uma mensagem muito grande podemos utilizar as aspas triplas (*''' ou '''*)

Exemplo:

```
print(''Olá, tudo bem ? Meu nome é Higor e eu tenho 21 anos de idade'')
```

## CORTAR/PULAR LINHA NO PYTHON

Às vezes precisamos pular a linha em nossa mensagem, a fim de facilitar o entendimento para o usuário e até mesmo para nós desenvolvedores, desta forma o texto fica mais visível a todos.

```
>>> print("Olá \nMundo")  
Olá  
Mundo  
>>>
```

---

## CAPÍTULO 7: CONDIÇÕES EM PYTHON

Em toda linguagem de programação precisamos de condições para tomarmos uma decisão.

Condições nada mais são, do que sentenças para descobrir se a lógica é VERDADEIRA ou FALSA.

Usando o mesmo cenário onde você é professor e quer descobrir se o aluno **A** ficou de recuperação ou não, você executa seu script em Python chamado *notaAluno.py* insere as notas de seu aluno e no final o seu verifica a média e exibe uma das mensagens abaixo:

**O Aluno A, está de recuperação.**

**O Aluno A, não está de recuperação.**

Para fazermos o programa acima precisamos de condições, pois se a média do aluno **A** for menor que **6** será exibida a mensagem:

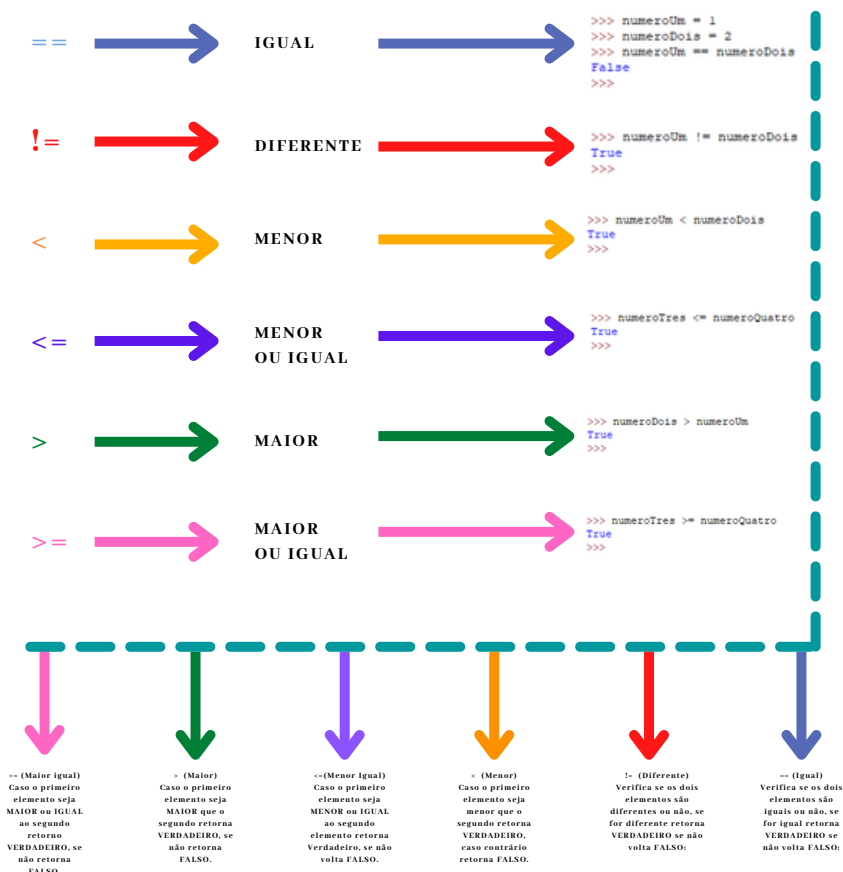
O Aluno **A**, está de recuperação.

Porém se for maior ou igual a **6** será exibida a seguinte mensagem:

O Aluno **A**, não está de recuperação.

Para sabermos se o Aluno **A** está ou não de recuperação, precisamos de sinais de comparação, pois **SE** a nota for **MAIOR** ou **IGUAL** a **6** o Aluno **A** não está de recuperação, mas **SE** a nota for **MENOR** que **6** ele está.

No Python temos as seguintes comparações:



Além dos principais mencionados acima, também existem alguns poucos usados como:

**IS:** igualdade.

Exemplo:

*a is b*

**IS NOT:** não igualdade.

Exemplo:

*a is not b.*

## DIFERENÇA ENTRE == & IS != & IS NOT

Se analisarmos BEM, existe uma semelhança clara entre sinal de == e IS e também != e IS NOT, para o entendimento deste conceito é necessário que você tenha entendido a relação de atribuição em Python, já explicado anteriormente neste livro.

Vamos criar uma lista em Python chamada **x** e faremos uma ATRIBUIÇÃO deste conteúdo para uma variável chamada **y**:

```
>>> x = [1, 2, 3, 4]
>>> y = x
>>> x
[1, 2, 3, 4]
>>> y
[1, 2, 3, 4]
```

Como podemos ver ambas aparentam(e são) idênticas, correto ?

```
>>> x == y
True
```

Sabemos que **Y** atribuiu o conteúdo de **X**, pois fizemos isto anteriormente, mas e se não soubessemos ?

Como iríamos descobrir que **Y** está referenciando o objeto, ou seja, o mesmo espaço na memória de **X**?

Por isso usamos o operador **IS**.

Em nosso exemplo anterior, **Y** está referenciando **X**, então:

```
>>> x is y  
True
```

Vamos criar agora uma outra lista chamada **C**, com o mesmo conteúdo também de **X**:

```
>>> c = [1, 2, 3, 4]
```

Agora faremos a comparação novamente, mas ao invés de **X** e **Y**, compararemos **X** e **C**:

```
>>> x == c  
True  
>>> x is c  
False
```

Como podemos ver, os elementos de **X** é igual a **C**, porém **C** não foi atribuído pela referência de **X**, por este motivo o retorno de "**X is C**" é **False**.

# O COMANDO IF

O comando **IF** é utilizado para COMPARAÇÃO entre dois elementos ou mais elementos, sua estrutura é definida da seguinte forma:

```
if condicao1 == condicao2:
    faça algo
elif condicao3 != condicao4:
    faça algo
elif condicao5 >= condicao6:
    faça algo
elif condicao7 <= condicao8:
    faça algo
else:
    faça algo
```

Imagine que as condições **IFs** são limitadas a serem processadas somente se o valor da comparação for igual a VERDADEIRO, enquanto ele não encontra uma comparação verdadeira, ele passará em todas as sentenças **ELIF**(que significa senão se em **Python**), se caso ele não encontrar **VERDADEIRO** em nenhuma sentença o processo cairá na lógica do **ELSE**.

Tá, eu acho que ficou meio confuso...  
Mas vamos lá...

Vamos voltar no exemplo do professor que quer validar a nota do Aluno A, beleza ?

```
nomeAluno      = input("Qual o nome do Aluno? ")
notaUmAluno    = int(input("Digite a nota 1 do Aluno? "))
notaDoisAluno  = int(input("Digite a nota 2 do Aluno? "))

notaMediaAluno = (notaUmAluno + notaDoisAluno)/2

if notaMediaAluno == 10:
    print("O aluno %s teve a média 10, dê os parabéns a ele." %(nomeAluno))
elif notaMediaAluno >= 6:
    print("Aluno %s Aprovado com a média de %d!" %(nomeAluno, notaMediaAluno))
else:
    print("Aluno %s Reprovado com a média de %d!" %(nomeAluno, notaMediaAluno))
```

Explicando cada trecho do programa acima:

```
if notaMediaAluno == 10:
```

Caso o aluno tenha a média de 10, então primeira condição será VERDADEIRA, desta forma será lido a mensagem:

```
print("O aluno %s teve a média 10, dê os parabéns a ele.", %(nomeAluno))
```

Caso a média do aluno tenha sido um valor DIFERENTE que 10, a primeira condição será FALSA, desta forma será lido a segunda sentença.

```
elif notaMediaAluno >= 6:
```

Se a média do aluno for MAIOR ou IGUAL a 6, então a condição será VERDADEIRA, e a seguinte mensagem será exibida:

```
print("Aluno %s Aprovado com a média de %d!", %(nomeAluno, notaMediaAluno))
```

Caso não encontre ele buscará a próxima condição, porém neste caso não existe uma próxima condição, somente o **ELSE**, ou seja, se entre todas as condições anteriores não existiu nenhuma **VERDADEIRA** sempre cairá no **ELSE**.

```
print("Aluno %s Reprovado com a média de %d!" , %(nomeAluno, notaMediaAluno))
```

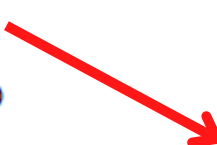
## OPERADORES AND & OR

Imagine que você queira validar se 2 variáveis obedecem uma condição, caso contrário você ignora e sair do programa. Para isso podemos utilizar o operador AND(E).

Se caso **TODAS** as sentenças forem **VERDADEIRAS** em sua **CONDIÇÃO**, o processo é marcado como **TRUE**, exemplo:

```
variavel1 = 1
variavel2 = 2

if variavel1 == 1 and variavel2 == 2:
    print(True)
else:
    print(False)
```



O PROGRAMA  
CAIRÁ AQUI

Porém se uma das validações forem falsas o processo é tido como FALSE.

```
variavel1 = 1
variavel2 = 2

if variavel1 == 1 and variavel2 == 3:
    print(True)
else:
    print(False)
```



O PROGRAMA  
CAIRÁ AQUI

Caso você queira validar se no MÍNIMO UMA condição seja VERDADEIRA é necessário utilizar o operador OR(OU).

```
if variavel1 == 1 or variavel2 == 3:
    print(True)
else:
    print(False)
```



O PROGRAMA  
CAIRÁ AQUI

Saber qual operador usar é muita pela situação e lógica, mas sempre traduza ao pé da letra, se caso você precisa que TODAS as sentenças de sua condição sejam verdadeiras e obrigatórias você usará AND, caso contrário usará OR.

Em casos específicos usará inclusive os dois.

## **Exercício Algoritmo**

Faça um programa em Python que peça o nome, idade e sexo da pessoa(M/F).

Logo após isso imprima o fase da vida em que esta pessoa se encontra:

**Para Homens[M]:**

- Menor ou Igual a 12 anos: INFÂNCIA
- até 22 anos: ADOLESCÊNCIA
- até 59 anos: ADULTA
- Maior ou igual a 60 anos: IDOSA.

**Para Mulheres[F]:**

- Menos que 12 anos: INFÂNCIA
- até 18 anos: ADOLESCÊNCIA
- até 62 anos: ADULTA
- Maior ou igual a 63 anos: IDOSA.

No final do exercício imprima para o usuário uma mensagem como:

<nome>, você tem <idade> e está na fase <fase>

**OBS:** Considerar M para Homens e F para Mulheres e apenas números Inteiros para a idade.

## CAPÍTULO 8:

# LAÇOS DE REPETIÇÃO

Agora aprenderemos um conceito muito interessante e muito usado em qualquer linguagem de programação chamado LAÇOS DE REPETIÇÃO, seguimos com o mesmo exemplo do professor no tópico passado.

Imagine que você tem uma sala com 45 alunos e precisa de um programa que mostre a você o número de quantos alunos foram aprovados e quantos reprovaram de ano.

No cenário acima, somente utilizando as condições mencionadas no tópico passado, isso seria muito custoso e inviável pois você teria que fazer várias validações de

*IF's.*

```
cont = 0
notaAluno1 = int(input("Nota do Aluno 1: "))
notaAluno2 = int(input("Nota do Aluno 2: "))
notaAluno3 = int(input("Nota do Aluno 3: "))
notaAluno4 = int(input("Nota do Aluno 4: "))

if notaAluno1 >= 6:
    cont += 1
if notaAluno2 >= 6:
    cont += 1
if notaAluno3 >= 6:
    cont += 1
if notaAluno4 >= 6:
    cont += 1

print("Quantidade de alunos aprovados: %d" % (cont))
```

Bom, fiz uma miniatura do código BEM “ruim” que programadores que não entendem de algoritmo fariam, imagina repetir isso para 45 alunos, o quanto trabalho daria ?

Além de quando entrasse ou saísse um aluno, você teria que trocar seu código, removendo ou adicionando uma nova condição para o aluno.

Acho que deu para entender a importância que um laço de repetição possui, vamos dar uma olhada na sintaxe do **FOR** em **Python**:

## COMANDO FOR

```
funcionarios = ["Higor", "Samara", "Maria"]  
  
for i in funcionarios:  
    print(i)
```

- “**FOR**” é o laço de repetição no **Python**,
- “**i**” é a variável que percorrerá cada posição do que vem depois do “in”
- “funcionários” é nossa lista criada.
- O método “**print()**” exibe cada elemento selecionado e processado no comando **for**.

```
C:\Users\Higor Lopes\Desktop>python teste2.py  
Higor  
Samara  
Maria
```

A expressão **FOR** lê todos os índices e posições UM A UM em um elemento, seja ele string, tuplas, lista e etc...

E armazena em uma variável, que em nosso caso é a *"i"*, mas poderia ser qualquer outra:

```
funcionarios = ["Higor", "Samara", "Maria"]

for funcionario in funcionarios:
    print(funcionario)
```

```
C:\Users\Higor Lopes\Desktop>python teste2.py
Higor
Samara
Maria
```

```
funcionarios = ["Higor", "Samara", "Maria"]

for i in funcionarios:
    print(i)
```

Podemos também mostrar os índices de cada funcionário, dentro de uma lista:

---

```
funcionarios = ["Higor", "Samara", "Maria"]

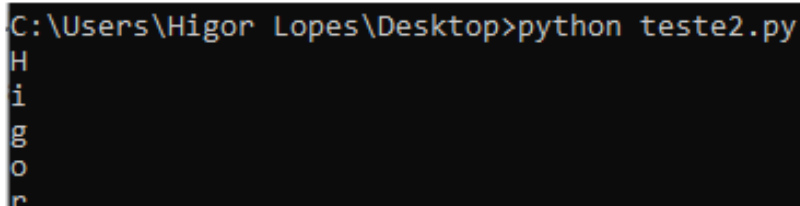
for index, funcionario in enumerate(funcionarios):
    print(index, funcionario)
```

```
C:\Users\Higor Lopes\Desktop>python teste2.py
0 Higor
1 Samara
2 Maria
```

Também podemos percorrer as posições de um texto(String) com **Python** usando laços de repetição:

```
nome = "Higor"

for i in nome:
    print(i)
```

A screenshot of a Windows command prompt window. The command 'C:\Users\Higor Lopes\Desktop>python teste2.py' has been entered. The output of the script is displayed on the next line, showing the characters 'H', 'i', 'g', 'o', and 'r' on separate lines.

```
C:\Users\Higor Lopes\Desktop>python teste2.py
H
i
g
o
r
```

## Laços de Repetição através de uma sequência de números.

Podemos utilizar a função **range()**, ela gerará uma lista de números para nós, esta função possui três parâmetros de entrada:

**range(start, end, step)**

Se o valor do primeiro parâmetro for igual a vazio ou não preenchido, será considerado como zero.

Vamos seguir alguns exemplos desta função:

```
range(5)
#resultado será uma lista com os elementos [0,1,2,3,4]
#posição final: 5

range(3,10)
#resultado será uma lista com os elementos [3,4,5,6,7,8,9]
#posição inicial: 3
#posição final: 10

range(4,10,2)
#resultado será uma lista com os elementos [4,6,8]
#posição inicial é: 4
#posição final é: 10
#porém o último parâmetro diz que deve pular de DOIS em DOIS, logo
#é desconsiderado o conteúdo de 5,7,9
```

se passarmos a função **range()** no **for**:

```
for i in range(10):
    print(i)
```

```
C:\Users\Higor Lopes\Desktop>python teste2.py
0
1
2
3
4
5
6
7
8
9
```

Vamos seguir alguns exemplos desta função:

```
range(5)
#resultado será uma lista com os elementos [0,1,2,3,4]
#posição final: 5

range(3,10)
#resultado será uma lista com os elementos [3,4,5,6,7,8,9]
#posição inicial: 3
#posição final: 10

range(4,10,2)
#resultado será uma lista com os elementos [4,6,8]
#posição inicial é: 4
#posição final é: 10
#porém o último parâmetro diz que deve pular de DOIS em DOIS, logo
#é desconsiderado o conteúdo de 5,7,9
```

se passarmos a função **range()** no **for**:

```
for i in range(10):
    print(i)
```

```
C:\Users\Higor Lopes\Desktop>python teste2.py
0
1
2
3
4
5
6
7
8
9
```

# COMANDO WHILE

Além do **FOR**, temos o **WHILE**, que também é um laço de repetição, ambos têm a mesma finalidade.

Enquanto a condição do **WHILE** ser **VERDADEIRA** ele continuará fazendo seu processo repetidamente, quando a condição torna-se **FALSA** seu processo é parado.

Imagine o cenário que você quer inserir uma sequência de números até que a soma deles seja menor que 100, podemos fazer isto da seguinte forma:

```
total = 0

while total < 100:
    num = int(input("Insira um valor: "))
    total += num

print("Limite máximo foi alcançado!")
```

```
C:\Users\Higor Lopes\Desktop>python teste2.py
Insira um valor: 10
Insira um valor: 10
Insira um valor: 10
Insira um valor: 10
Insira um valor: 40
Insira um valor: 1
Insira um valor: 20
Limite máximo foi alcançado!
```

Na maioria das linguagens nós precisamos criar um contador, para que algumas lógicas possam ser possíveis de serem executadas e também possamos ter controle total dentro dos nossos laços de repetição.

```
cont = 10

while cont > 0:
    print("Contador = ", cont)
    cont -= 1
```

```
C:\Users\Higor Lopes\Desktop>python teste2.py
Contador = 10
Contador = 9
Contador = 8
Contador = 7
Contador = 6
Contador = 5
Contador = 4
Contador = 3
Contador = 2
Contador = 1
```

### ATENÇÃO:

Todas as vezes que você fizer um laço de repetição, tome cuidado, pois se ele não encontrar nenhuma condição FALSE, o mesmo ficará rodando eternamente.

 Prompt de Comando - python teste2.py[illegible]

## COMANDO BREAK

### Exemplo:

```
teste.py - C:/Users/Higor Lopes/Desktop/teste.py (3.7.9)
File Edit Format Run Options Window Help
a = 0
for i in range(5):
    a += 2
    print('i = ', i, '/ a = ', a)
    if a == 6:
        break
```

## COMANDO CONTINUE

Outra palavra chave útil é o **CONTINUE**, sempre quando usado dentro de um laço de repetição é ignorado TUDO que vem após ele, ou seja, como o próprio nome sugere ele continua o processo voltando para a leitura do próximo registro do laço de repetição.

```
teste.py - C:/Users/Higor Lopes/Desktop/teste.py (3.7.9)
File Edit Format Run Options Window Help
a = 0
for i in range(5):
    a += 2
    if a <= 6:
        continue

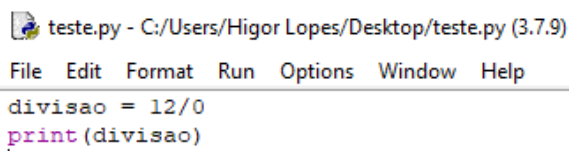
    print('i = ', i, '/ a = ', a)
    break
```

O **CONTINUE** volta para o comando FOR e ignora todo o processamento restante, já o comando **BREAK** sai do laço de repetição.

## CAPÍTULO 9: TRY & EXCEPT

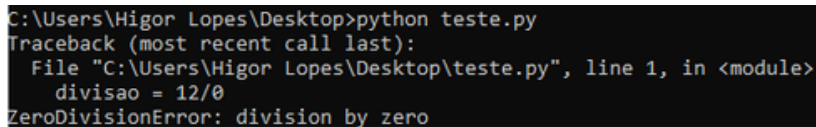
Este comando controla como deve ser feito o processo do programa e como o próprio nome diz, **TENTA** fazer algo, mas se ocorrer algum erro ele processará a **EXCEÇÃO**.

Exemplo simples e claro:



```
teste.py - C:/Users/Higor Lopes/Desktop/teste.py (3.7.9)
File Edit Format Run Options Window Help
divisao = 12/0
print(divisao)
```

Desta forma ao executarmos o programa ocorrerá o seguinte erro:



```
C:\Users\Higor Lopes\Desktop>python teste.py
Traceback (most recent call last):
  File "C:\Users\Higor Lopes\Desktop\teste.py", line 1, in <module>
    divisao = 12/0
ZeroDivisionError: division by zero
```

Como pode ser visto o erro ocorrido foi “standard”, ou seja, um erro padrão do próprio interpretador do **Python** que sugere inclusive uma solução do problema, existem várias mensagens pré definidas, segue abaixo algumas delas:

- IOError
- ImportError
- IndexError
- KeyError
- NameError

Mas se nós utilizássemos o **TRY EXCEPT** (ou **TRY CATCH** em outras linguagens) podemos pré-definir esta mensagem de erro para que toda vez que o interpretador encontrar um problema seja lido o seu **EXCEPT**:

 teste.py - C:/Users/Higor Lopes/Desktop/teste.py (3.7.9)

File Edit Format Run Options Window Help

```
try:
    divisao = 12/0
    print(divisao)
except:
    print("Não pode ser feito divisão por 0")
```

```
C:\Users\Higor Lopes\Desktop>python teste.py
Não pode ser feito divisão por 0
```

# CAPÍTULO 10:

## FUNÇÕES & MÓDULOS

### O QUE SÃO FUNÇÕES ?

A grosso modo, funções nada mais são do que códigos segmentados que são chamados uma ou mais vezes em seu programa, muito utilizado para segmentar blocos de códigos para a reutilização.

Suponhamos que em nosso programa nós precisamos validar o CPF do usuário várias vezes, imagina só termos que repetirmos o mesmo código para cada situação ?

Em **Python**, as funções possuem parênteses e dentro deles são inseridos os parâmetros, uma função pode ter nenhum ou vários parâmetros.

Um exemplo disso é a função **replace()**, ela manipula o texto de uma **string**, exemplo:

```
>>> meuNome = "Higor Lopes"
>>> meuNome.replace("Lopes", "Granero")
'Higor Granero'
>>> meuNome
'Higor Lopes'
```

Como pode ser visto, foi chamada a função **replace()** com dois parâmetros e ela retorna a nós o valor alterado.

## DEFININDO UMA FUNÇÃO

Agora que vimos a definição das funções em **Python**, podemos criar nossas próprias funções, sua sintaxe é:

```
def nomeFuncao(parametros):  
    return
```

**def** é o que define que a próxima expressão será uma função, como vimos toda função pode ter nenhum ou vários parâmetros, por último o **return** está relacionado ao retorno de dados de sua função, uma função pode ou não retornar dados, isto depende de sua necessidade.

Exemplo de uma função para a soma de dois números:

```
try:  
    n1 = int(input("Digite o Primeiro Número: "))  
    n2 = int(input("Digite o Segundo Número: "))  
  
    def nomeFunc(numero1, numero2):  
        resultado = numero1 + numero2  
        return print("O Resultado é: %d" %(resultado))  
  
    nomeFunc(n1, n2)  
  
except:  
    print("É permitido apenas números Inteiros")
```

Como pode ser visto pedimos ao usuário para entrar com dois valores, foi definida a função com dois parâmetros de entrada e com um retorno mostrando o resultado da soma.

Uma curiosidade e atenção que devemos ter com as funções é que toda variável criada dentro dela não está visível para o resto do programa, aqui entramos nos assuntos de variável local e variável global, a primeira só existe dentro da função, podemos refletir isto com a variável resultado, olha o exemplo abaixo, Se tentarmos acessar esta variável fora da função, ocorre um erro:

```
n1 = int(input("Digite o Primeiro Número: "))
n2 = int(input("Digite o Segundo Número: "))
```

```
def nomeFunc(numero1, numero2):
    resultado = numero1 + numero2
    return None
```

```
nomeFunc(n1, n2)
print("O Resultado é: %d" %(resultado))
```

```
Digite o Primeiro Número: 10
Digite o Segundo Número: 10
Traceback (most recent call last):
  File "C:\Users\Higor Lopes\Desktop\teste.py", line 9, in <module>
    print("O Resultado é: %d" %(resultado))
NameError: name 'resultado' is not defined
```

Variáveis globais ficam fora das funções e podem ser chamada para todo o programa.

# FUNÇÕES PRONTAS

Você lembra dos conversores,

- `int()`
- `float()`
- `str()` ?

Pois é, eles são funções prontas para uso, ou seja, já nativas para qualquer programa Python, sem a necessidade de importarmos nada.

Separei aqui algumas outras funções nativas:

`abs(x)` .....> Retorna o valor absoluto do número `x` (converte números negativos em positivos)

`bin(x)` .....> Retorna uma string que representa o valor de `x` convertido em binário

`format(x,y)`.....> Retorna `x` formatado conforme direcionado pela string de formato `y`. No Python moderno, você é mais provavelmente usará strings `f`, conforme descrito posteriormente neste capítulo

`hex(x)`.....> Retorna uma string contendo `x` convertido em hexadecimal, prefixado com `0x`

`max(x,y,z ...)` .....> Obtém qualquer número de argumentos numéricos e retorna o que for maior.

`min(x,y,z ...)`.....> Obtém qualquer número de argumentos numéricos e retorna o que for MENOR.

`oct(x)`.....> Converte `x` em um número octal, prefixado com `0o` para indicar octal.

`round(x,y)`.....> Arredonda o número `x` para o número `y` de casas decimais

`type(x)` .....> Retorna uma string indicando o tipo de dados de `x`

# MÓDULOS

**Python** vem com um grande número de módulos integrados.

Um módulo possui várias funções dentro dele e para usarmos estas funções precisamos primeiro importar em nosso programa, pois não estão disponíveis nativamente.

Nós fazemos isso usando a palavra-chave *import*.

Existem três maneiras de fazer isso:

```
import random
```

```
import random as r
```

```
from random import randrange
```

Deste modo, você importa um módulo inteiro para o seu programa, se ele tiver várias funções dentro dela, **TODAS** elas ficam acessíveis e para utilizá-las é necessário apenas referenciar:

```
random.randrange(2, 100)
```

Deste modo é mais específico e você referência à função específica que usará do módulo importado.

```
random.randrange(2, 100)
```

Deste modo você importa todo o módulo para seu programa, assim como no primeiro caso, porém é referenciado um apelido para o módulo, neste caso "r".

```
r.randrange(2, 100)
```



## QUAIS OS PRINCIPAIS MÓDULOS EXISTENTES ?

Existem vários módulos e consequentemente infinitas funções, seria basicamente impossível mencionar todos neste livro.

Porém separei alguns módulos mais importantes:

### MATEMÁTICA EM PYTHON MATH

| Method                    | Description                                         |
|---------------------------|-----------------------------------------------------|
| <code>math.acos()</code>  | Retorna o arco cosseno de um número                 |
| <code>math.acosh()</code> | Retorna o cosseno hiperbólico inverso de um número  |
| <code>math.asin()</code>  | Retorna o arco seno de um número                    |
| <code>math.asinh()</code> | Retorna o seno hiperbólico inverso de um número     |
| <code>math.atan()</code>  | Retorna o arco tangente de um número em radianos    |
| <code>math.atan2()</code> | Retorna o arco tangente de $y / x$ em radianos      |
| <code>math.atanh()</code> | Retorna a tangente hiperbólica inversa de um número |
| <code>math.ceil()</code>  | Arredonda um número para o inteiro mais próximo     |

[https://www.w3schools.com/python/module\\_math.asp](https://www.w3schools.com/python/module_math.asp)

Como vimos, para utilizarmos as funções do módulo `math()` precisamos importá-los:

```
>>> import math
>>> math.pi
3.141592653589793
```

## DATAS EM PYTHON DATETIME

Módulo responsável pelo controle de datas, alguns exemplos do seu uso:

```
>>> import datetime
>>> x = datetime.datetime.now()
>>> print(x)
2021-01-26 02:50:06.616576
```

```
>>> x = datetime.datetime(2020, 5, 17)
>>> print(x)
2020-05-17 00:00:00
```

## CRIANDO UM MÓDULO

Além de importar módulos prontos, também podemos criar nossos próprios módulos.

Isso é muito útil se você tiver algumas funções que deseja reutilizar em outros projetos de programação.

Criar um módulo é simples, basta salvar o arquivo com a extensão *.py* e coloque-o na mesma pasta do arquivo *Python* para o qual você vai importar.

Então suponha que você queira usar a função *checarPrimo()* para definir se o número é primo ou não.

Primeiro salve este código como *primoNumero.py* em sua área de trabalho com o seguinte código:

```
def checarPrimo(numero):  
    for i in range(2, numero):  
        if (numero%i == 0):  
            return False  
    return True
```

Depois disso você cria um novo arquivo *usarFuncChecarPrimo.py* com o seguinte código:

```
import primoNumero
resposta = primoNumero.checarPrimo(3)
print(resposta)
```

Se rodar este código a resposta será True, pois “3” é um número primo.

**ATENÇÃO: SÓ FUNCIONARÁ SE AMBOS OS ARQUIVOS ESTIVEREM NA MESMA PASTA.**

No entanto, suponha que você queira armazenar

*checarPrimo.py* e *usarFuncChecarPrimo.py* em pastas diferentes. Você vai ter que adicionar alguns códigos para *usarFuncChecarPrimo.py* para informar ao interpretador Python onde encontrar este módulo.

Digamos que você tenha criado uma pasta chamada “**ModulosPython**” em sua unidade “C:” do Windows e armazenou o programa *checarPrimo.py*.

Você precisa adicionar o seguinte código no topo do seu arquivo *usarFuncChecarPrimo.py* para referenciar o programa *checarPrimo.py* que encontra-se na pasta **ModulosPython** (Antes do *import* de *checarPrimo.py*)

```
import sys

if 'C:\\ModulosPython' not in sys.path:
    sys.path.append('C:\\ModulosPython')
```

*sys.path* se refere ao caminho do seu sistema **Python**.

Esta é a lista de diretórios que o **Python** percorre para procurar módulos e arquivos. o código acima anexa a pasta 'C:\\**ModulosPython**' para que o Python enxergue esta pasta também.

Agora você pode importar *checarPrimo.py* em qualquer programa.

Criar módulo é algo útil para projetos maiores.

# CAPÍTULO 11:

## EXERCÍCIOS RESOLVIDOS

### 1) Converta Minutos para Segundos:

Exemplo:

|                            |   |     |
|----------------------------|---|-----|
| <code>converter(5)</code>  | → | 300 |
| <code>converter(10)</code> | → | 600 |
| <code>converter(15)</code> | → | 900 |

Resolução

```
def converter(minutos):  
    return minutos * 60  
  
print(converter(15))
```

### 2) Converta Minutos para Segundos:

Exemplo:

|                            |   |     |
|----------------------------|---|-----|
| <code>converter(5)</code>  | → | 300 |
| <code>converter(10)</code> | → | 600 |
| <code>converter(15)</code> | → | 900 |

Resolução

## **CAPÍTULO 12:**

### **DESAFIOS**

**1. Faça um Programa em Python para converter a temperatura em graus centígrados para Fahrenheit.**

**2. Faça um Programa Python para encontrar a soma dos dígitos de um inteiro usando o loop while**

**3. Faça um Programa Python para exibir todos os múltiplos de 2 em um intervalo de valores inseridas pelo usuário.**

- 4.Faça um Programa Python para verificar se o número inteiro digitado pelo usuário é um número primo ou não.**
- 5.Faça um Programa Python para imprimir os números de um determinado número inserido pelo usuário até 0 usando recursão.**
- 6.Programa Python para encontrar o maior número em uma lista sem usar funções integradas**
- 7.Programa Python para inserir um número em qualquer posição em uma lista.**
- 8. Faça um Programa Python para excluir um elemento de uma lista por índice**
- 9. Programa Python para verificar o ano bissexto**
- 10.Programa Python para imprimir todos os itens de um dicionário**